Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

К защите допустить:

Заведующий кафедрой ИИТ

_____ Д. В. Шункевич

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовому проекту
по дисциплине «Модели решения задач в интеллектуальных системах»
на тему:

ИНТЕРФЕЙС ИНТЕЛЛЕКТУАЛЬНОЙ СИСТЕМЫ АВТОМАТИЧЕСКОЙ ПРОВЕРКИ СТУДЕНЧЕСКИХ РАБОТ

БГУИР КП6 6-05-0611-03 039 ПЗ

Студент гр. 321702 П. А. Котко
Проверяющий Н. В. Гракова

Минск 2026

СОДЕРЖАНИЕ

Перечень условных обозначений 2
Введение 3
1 Анализ подходов к разработке интерфейса интеллектуальной системы автоматизации проверки студенческих работ 5
1.1 Характеристика предметной области 5
1.2 Типы решаемых задач и ожидаемые результаты 6
1.3 Ограничения и особенности системы 6
1.4 Ролевая модель и задачи участников 7
1.5 Функциональные возможности ролей 8
1.6 Взаимодействие с внешними и внутренними подсистемами . 9
1.7 Анализ существующих систем и интерфейсов 9
1.8 Анализ требований к интерфейсу 13
1.9 Вывод 15
2 Проектирование интеллектуальной системы 17
2.1 Общая архитектура интеллектуальной системы 17
2.2 Архитектура интерфейсного слоя 18
2.3 Модели пользователей и сценарии использования 20
2.4 Проектирование пользовательского интерфейса 23
2.5 Проектирование программного интерфейса 24
2.6 Проектирование аппаратного интерфейса 25
2.7 Проектирование интеллектуальных механизмов интерфейса 25
2.8 Вывод по разделу проектирования 26
3 Реализация интеллектуальной системы 28
3.1 Выбор средств реализации 28
3.2 Реализация программного (или аппаратного) интерфейса . . 28
3.3 Тестирование интерфейса 28
3.4 Вывод по разделу реализации 28
Заключение 29
Список использованных источников 30

ПЕРЕЧЕНЬ УСЛОВНЫХ ОБОЗНАЧЕНИЙ

LMS (Learning Management System) — система управления обучением.

NLP (Natural Language Processing) — обработка естественного языка.

ИИ — искусственный интеллект.

API (Application Programming Interface) — программный интерфейс приложения.

ГОСТ — государственный стандарт.

PDF (Portable Document Format) — формат электронных документов.

LLM (Large Language Model) — большая языковая модель.

GUI (Graphical User Interface) — графический интерфейс пользователя.

ВВЕДЕНИЕ

В условиях цифровизации образования и перехода к смешанным формам обучения нагрузка на преподавательский состав существенно возрастает. Одной из наиболее трудоемких задач является оперативная и объективная проверка самостоятельных работ. Традиционные методы контроля не позволяют обеспечить мгновенную обратную связь, что снижает эффективность усвоения материала студентами.

Актуальность разработки интерфейса интеллектуальной системы (ИС) обусловлена необходимостью создания удобного и интуитивно понятного инструмента взаимодействия между сложными алгоритмами анализа данных и конечным пользователем. В контексте современной инженерии знаний интерфейс перестает быть просто визуальной оболочкой и становится средой когнитивной поддержки, позволяющей корректно интерпретировать результаты работы ИИ. Данный проект является логическим продолжением изучения методов искусственного интеллекта и проектирования сложных программных систем, начатого в предыдущих семестрах, перенося фокус на человеко-ориентированный дизайн (HCD) в образовательной среде.

Целью курсового проекта является проектирование и разработка прототипа пользовательского интерфейса интеллектуальной системы, предназначенной для автоматизированного анализа и оценки студенческих работ.

Для достижения поставленной цели необходимо решить следующие задачи:

– провести анализ предметной области и существующих аналогов систем проверки знаний;

– сформировать требования к функциональности и эргономике интерфейса на основе сценариев взаимодействия;

– разработать архитектуру интерфейса и информационную структуру системы;

– создать визуальную концепцию и макеты экранных форм (прототип);

– оценить эффективность спроектированных интерфейсных решений.

Объект разработки — интеллектуальная система автоматической проверки студенческих работ.

Предмет разработки — пользовательский интерфейс, включая сценарии взаимодействия (UX), архитектуру переходов и визуальное воплощение компонентов управления (UI).

Границы разработки — работа ограничивается проектированием фронтенд-части и логики взаимодействия пользователя с системой. Реализация серверных алгоритмов глубокого анализа текста (NLP) или нейросетевых моделей оценки в рамках данного проекта рассматривается как «черный ящик» с заданными входными и выходными интерфейсами. Про-

тотип включает в себя основные модули: личный кабинет, панель загрузки работ и экран визуализации результатов проверки системы.

1 АНАЛИЗ ПОДХОДОВ К РАЗРАБОТКЕ ИНТЕРФЕЙСА ИНТЕЛЛЕКТУАЛЬНОЙ СИСТЕМЫ АВТОМАТИЗАЦИИ ПРОВЕРКИ СТУДЕНЧЕСКИХ РАБОТ

1.1 Характеристика предметной области

Предметной областью исследования является процесс автоматизированного контроля и оценки знаний в цифровой образовательной среде. В современных условиях системы дистанционного обучения (LMS) и специализированные платформы переходят от простейших тестовых форм к интеллектуальному анализу свободных ответов, эссе и программного кода.

Данная тенденция обусловлена резким увеличением объема цифрового документооборота в учебных заведениях. Проверка курсовых и дипломных работ вручную становится важной составляющей в образовательном процессе. Преподаватель вынужден тратить значительную часть времени не на оценку содержания и научной новизны, а на проверку формальных требований: соблюдение шрифтов, отступов, правил цитирования и государственных стандартов (нормоконтроль).

В рамках этой области автоматизация контроля разделяется на два ключевых направления:

1. **Семантический контроль** — анализ смысла текста и логики изложения.
2. **Технический контроль (нормоконтроль)** — проверка структуры документа и его соответствия регламентированным правилам оформления (ГОСТ, методические указания).

Система SAPSR фокусируется на втором направлении, автоматизируя наиболее рутинные операции. Интеллектуальная составляющая таких систем заключается в способности алгоритмов распознавать элементы документа (заголовки, таблицы, списки литературы) и сопоставлять их с эталонными требованиями, хранящимися в базе знаний системы.

Интерфейс интеллектуальной системы в данной области выступает связующим звеном между сложными алгоритмами обработки данных (нейронными сетями, семантическим анализом) и конечным пользователем — преподавателем или студентом. Важной характеристикой предметной области является требование к интерпретируемости результатов: интерфейс должен не просто выдать вердикт «работа не принята», а визуально отобразить места нарушений в тексте.

Таким образом, использование микросервисной архитектуры и интеллектуальных модулей анализа позволяет создать гибкую среду, которая адаптируется под стандарты различных кафедр и вузов, обеспечивая объ-

ективность и высокую скорость проверки студенческих работ.

1.2 Типы решаемых задач и ожидаемые результаты

Функционирование интеллектуальной системы направлено на автоматизацию следующих категорий задач:

– **Оценка работ с открытой формой ответа:** анализ текстового задания на соответствие грамматическим нормам, ГОСТам.

– **Проверка введённых данных:** автоматическая проверка заполнения определённых ключевых точек в задании и проверка соответствия данных.

– **Адаптивное консультирование:** генерация персонализированных рекомендаций и подсказок по исправлению ошибок в режиме реального времени.

Ожидаемым результатом работы системы является не только итоговая оценка в баллах, но и детализированный отчет (*Feedback*), визуализирующий слабые стороны выполненной работы.

1.3 Ограничения и особенности системы

При проектировании интерфейса системы SAPSR учитывалось целый ряд условий: от технических возможностей платформы до этических особенностей образовательной среды. Эти факторы определяют, как именно пользователь взаимодействует с нашим сервисом.

Анализ среды развертывания и специфики интеллектуального анализа позволяет выделить следующие ограничения, которые необходимо учитывать при проектировании:

– **Ограничения платформы Telegram Mini Apps.** Использование мессенджера в качестве среды развёртывания накладывает жесткие рамки на эргономику: интерфейс должен быть адаптирован под портретную ориентацию (mobile-first), учитывать отсутствие стандартной браузерной навигации («Назад/Вперед») и поддерживать бесшовную авторизацию через передачу параметров initData.

– **Ограничение формата входных данных.** Система принимает документы исключительно в формате PDF. Это обусловлено тем, что данный формат гарантирует сохранение точных координат элементов (шрифтов, отступов) и метаданных символов, что критично для корректного нормоконтроля. Интерфейс должен обеспечивать валидацию типа файла на стороне клиента.

– **Асинхронный характер проверки.** Процесс анализа документа интеллектуальными алгоритмами (NLP и парсинг структуры) занимает от 5 до 30 секунд. Это накладывает требование к реактивности интерфейса:

использование индикаторов прогресса (Skeleton screens, Progress bars) и системы push-уведомлений, чтобы исключить состояние неопределенности для пользователя.

– **Функциональные границы.** В текущей реализации система автоматизирует только проверку формальных признаков (шрифты, поля, структура). Оценка смыслового наполнения остается прерогативой преподавателя. Интерфейс должен визуально разделять «технический вердикт системы» и «экспертную рецензию», исключая ложное восприятие автоматической оценки как финальной.

1.4 Ролевая модель и задачи участников

В процессе функционирования интеллектуальной системы автоматической проверки работ выделяются две основные группы активных участников (ролей), а также внешние и внутренние программные сущности, обеспечивающие реализацию бизнес-логики. В рамках проектируемой интеллектуальной системы выделяются следующие ключевые участники и их функциональные задачи:

1. **Студент** - основной пользователь, предоставляющий объект проверки. Как у любого участника системы, у студента есть цели и задачи:

– **Цель** - успешное прохождение автоматизированного нормоконтроля и получение экспертной оценки содержательной части работы.

– **Контекст использования** - интерфейс мессенджера Telegram.

– **Задачи:**

– прохождение первичной регистрации с привязкой аккаунта и вводом персональных данных (ФИО, группа);

– загрузка итераций курсовой работы в формате `.pdf`;

– анализ детализированных отчетов о технических ошибках оформления;

– мониторинг статуса проверки и получение финального вердикта.

2. **Преподаватель** - эксперт, осуществляющий финальную верификацию работы.

– **Цель** - проверка смыслового наполнения работ, предварительно отфильтрованных системой на соответствие техническим требованиям.

– **Контекст использования** - интерфейс мессенджера Telegram.

– **Задачи:**

– авторизация в системе с использованием индивидуального кода доступа;

– изучение логики и расчетов в верифицированных работах;

– вынесение решения («Принять» или «На доработку») с вводом текстовой рецензии.

3. **Внутренние и внешние подсистемы** - программные компоненты, обеспечивающие работу интерфейса

– **Модуль интеллектуального нормоконтроля** - выполняет парсинг структуры документа и сопоставление с эталонными требованиями.

– **Модуль бизнес-логики и маршрутизации** - "ядро"системы, написанное на Java. Он управляет процессом смены состояний и управления доступом.

– **Модуль асинхронного взаимодействия** - связывает Java-ядро и Python-сервисы.

– **Telegram API** - выступает в роли транспортного шлюза для взаимодействия со студентами.

– **Сервер базы данных** - обеспечивает целостность и хранение информации о пользователях, файлах, шаблонах и результатах проверки.

1.5 Функциональные возможности ролей

1. **Функционал роли «Студент»** - Интерфейс для данной роли ориентирован на оперативное взаимодействие и получение быстрой обратной связи. Основные функции включают:

– **Идентификация и профилирование** - Привязка Telegram-аккаунта к базе данных системы, ввод метаданных (ФИО, номер учебной группы) при первичном входе.

– **Управление объектами проверки** - Загрузка курсовых работ в формате `.pdf` с указанием целевой дисциплины и проверяющего преподавателя.

– **Интерпретация результатов нормоконтроля:**

– При наличии технических ошибок - получение детализированного протокола несоответствий (например, нарушение полей, отсутствие нумерации страниц).

– При успешном прохождении - получение подтверждения о передаче работы на этап экспертной оценки.

– **Мониторинг статуса** - Получение push-уведомлений об итоговом вердикте преподавателя (зачет или возврат на доработку).

2. **Функционал роли «Преподаватель»** - Интерфейс данной роли спроектирован как рабочее место эксперта, освобожденное от рутинных операций по проверке оформления.

– **Администрирование доступа** - Авторизация в системе с использованием уникального токена (кода доступа) для активации расширенных прав.

– **Агрегация данных (Dashboard)** - Просмотр единого списка работ, успешно прошедших интеллектуальный технический фильтр.

– **Экспертный анализ** - Доступ к скачиванию оригинальных файлов для анализа логики, расчетов и научной новизны представленных материалов.

– **Формирование вердикта** - Инструментарий для фиксации решения в системе:

– Статус «Принято» — завершение цикла проверки.

– Статус «Отправлено на доработку» — возврат работы студенту с обязательным текстовым обоснованием (рецензией).

1.6 Взаимодействие с внешними и внутренними подсистемами

Помимо конечных пользователей, участниками взаимодействия являются программные компоненты и внешние сервисы, обеспечивающие работу интерфейса:

1. **Модуль бизнес-логики и маршрутизации** - выступает в роли «ядра» системы, написанном на Java, управляет процессами смены состояний и разграничением прав доступа.

2. **Интеллектуальный модуль нормоконтроля** - анализирует внутреннюю структуру PDF-файла, проверяет соответствие параметров оформления ГОСТ и регламенту вуза.

3. **Модуль асинхронного взаимодействия** - обеспечивает связующее звено между Java-ядром и внешними Python-сервисами, гарантируя отзывчивость интерфейса.

4. **Telegram API** - обеспечивает транспортный уровень для передачи уведомлений и приема файлов от студентов в режиме реального времени.

5. **Сервер базы данных** - обеспечивает долгосрочное хранение и целостность информации о пользователях, загруженных файлах, шаблонах и результатах проверки.

1.7 Анализ существующих систем и интерфейсов

В рамках исследования был проведен анализ существующих программных решений, предназначенных для автоматизации проверки учебных работ и оценки образовательных результатов. В качестве наиболее репрезентативных аналогов были рассмотрены системы EssayGrader и Gradescope.

Обзор аналогичных систем

EssayGrader[1] - Система ориентирован на автоматизацию процесса проверки эссе и письменных работ с использованием алгоритмов искусственного интеллекта. Основное назначение системы — сокращение временных за-

трат преподавателя на проверку текстов при сохранении высокого качества обратной связи (анализ структуры, грамматики, стиля и аргументации).

1. **Тип интерфейса** - «человек–система». Взаимодействие осуществляется через веб-ориентированный графический интерфейс пользователя (GUI). Преподаватель загружает работы и настраивает параметры оценивания, а система возвращает детализированный отчет.

2. **Особенности** - Система предоставляет инструменты для генерации пояснительных комментариев, что позволяет автоматизировать рутинные аспекты рецензирования.

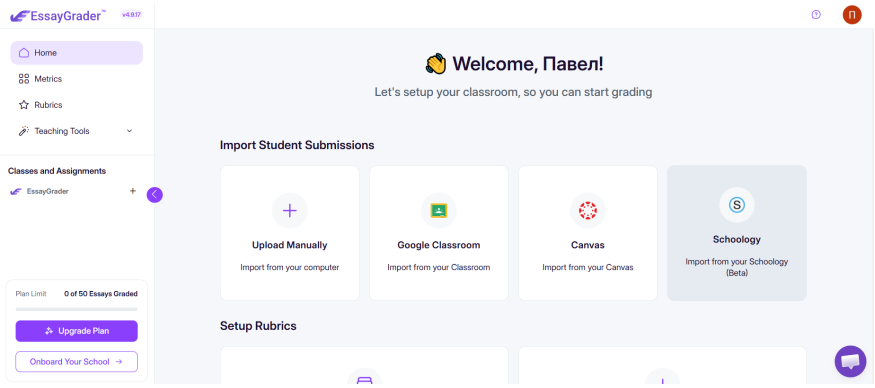

Рисунок 1.1 – EssayGrader

Gradescope[2] - представляет собой универсальный инструмент для оценки различных типов работ: от классических бумажных тестов и экзаменов до заданий по программированию. Система активно использует технологии машинного обучения для группировки похожих ответов, что позволяет оценивать сразу несколько работ одновременно.

1. **Тип интерфейса** - «человек–система». Основное взаимодействие преподавателей и студентов с веб-платформой для загрузки работ, ручного оценивания и просмотра результатов.

2. **Особенности** - сочетание ручного и автоматизированного ввода данных, а также поддержка инструментов для коллективной проверки работ в режиме реального времени.

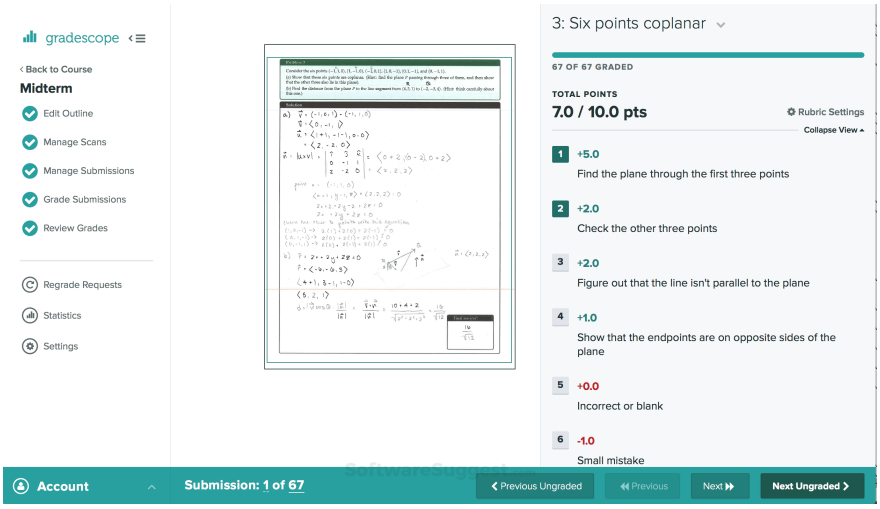

Рисунок 1.2 – Gradescope

EduMark[3] — универсальная платформа для оцифровки, проверки и оценивания учебных работ (как рукописных, так и цифровых) преподавателями. Система автоматизирует рутинные этапы проверки: группирует похожие ответы, применяет единые рубрики, проверяет код и ведет статистику успеваемости.

Тип интерфейса — человек–система. Преподаватель выступает в роли оператора: он загружает работы, настраивает рубрики, запускает функции группировки или авто-проверки кода и утверждает итоговые оценки. Студенты также взаимодействуют с системой через веб-интерфейс, чтобы загружать свои решения и просматривать результаты проверки.

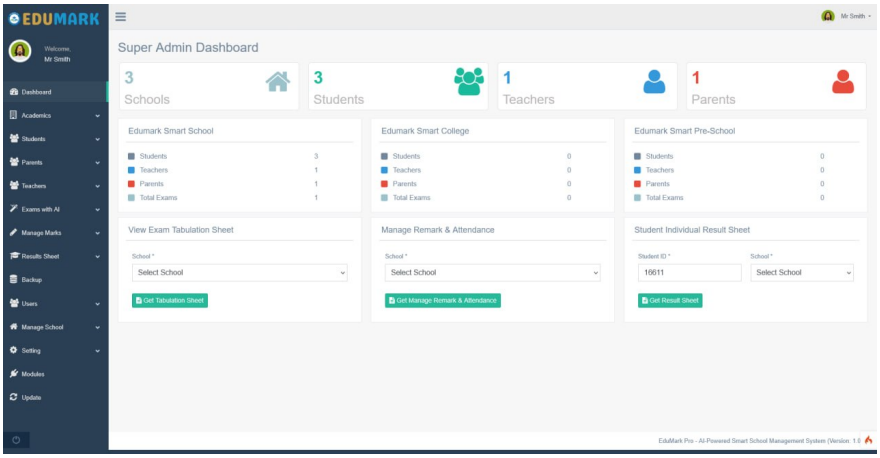

Рисунок 1.3 – Интерфейс системы EduMark

Анализ интерфейсов существующих решений

В данном подразделе проводится детальный анализ программных решений, выбранных в качестве аналогов. Особое внимание уделяется структуре взаимодействия пользователей с системами и эргономическим характеристикам их интерфейсов.

EssayGrader[1] — специализированный программный комплекс на базе искусственного интеллекта, предназначенный для автоматизации проверки эссе и предоставления детализированной обратной связи.

– **Ключевые сценарии и структура** - Типовой сценарий включает загрузку текстового файла, выбор критериев оценки (рубрик) и генерацию отчета. Структура интерфейса одноуровневая, сфокусированная на рабочем пространстве редактора и панели параметров ИИ.

– **Типовые операции** - Импорт документов, настройка уровня строгости проверки, редактирование сгенерированных ИИ комментариев, экспорт финальной рецензии.

– **Сильные стороны:**

  – **Удобство** - Минималистичный дизайн снижает когнитивную нагрузку на преподавателя.

  – **Скорость** - Моментальная генерация контекстного фидбека по сложным стилистическим ошибкам.

– **Слабые стороны:**

– **Прозрачность** - Алгоритмы оценки работают по принципу «черного ящика», что затрудняет апелляцию студента к конкретному баллу.

– **Гибкость** - Отсутствие инструментов для проверки нетекстовых данных (схем, формул).

Gradescope[2] — многофункциональная платформа для оценки широкого спектра учебных заданий: от рукописных работ до программного кода.

– **Ключевые сценарии и структура** - Сценарии варьируются от потокового сканирования бумажных бланков до автоматического тестирования программ (Autograder). Структура иерархическая: «Курс — Задание — Студент — Критерий».

– **Типовые операции** - Группировка похожих ответов с помощью ИИ, создание динамических рубрик (изменение балла во всех работах одновременно), синхронизация оценок с журналом успеваемости.

– **Сильные стороны:**

  – **Гибкость** - Поддержка различных форматов входных данных и возможность коллективной проверки.

  – **Прозрачность** - Инструмент «AI-assisted grading» группирует похожие ответы, обеспечивая единообразие и справедливость оценки.

– **Слабые стороны:**

  – **Ошибки проектирования** - Избыточность интерфейса при работе с малыми группами; высокий порог вхождения для настройки автоматических тестов кода (требуется написание скриптов конфигурации).

EduMark[3] — универсальная программная платформа, предназначенная для комплексной оцифровки, автоматизированной проверки и

оценивания учебных достижений в цифровой и рукописной формах.

– **Ключевые сценарии и структура** — основной сценарий включает загрузку массивов студенческих работ, настройку унифицированных рубрик оценивания и запуск процедур интеллектуальной группировки. Структура интерфейса ориентирована на ролевую модель «Преподаватель-Оператор», обеспечивая прямой доступ к инструментам массовой обработки данных.

– **Типовые операции** — пакетная загрузка файлов, автоматическое распознавание похожих фрагментов ответов, конфигурация параметров автопроверки программного кода, утверждение итоговых ведомостей и анализ статистики успеваемости.

– **Сильные стороны:**

  – **Эффективность** — значительное сокращение временных затрат на проверку за счет автоматизации рутинных этапов и группировки однотипных решений.

  – **Объективность** — применение единых рубрик и критериев ко всем работам минимизирует субъективный фактор при выставлении баллов.

– **Слабые стороны:**

  – **Сложность** — необходимость предварительной подготовки и детальной настройки рубрик перед началом процесса оценивания.

  – **Зависимость** — критическая роль качества исходных данных (сканов или файлов) для корректной работы алгоритмов интеллектуальной группировки.

1.8 Анализ требований к интерфейсу

На основе анализа предметной области и ролевых моделей участников взаимодействия сформулированы требования к проектируемому интерфейсу интеллектуальной системы.

Функциональные требования

Функциональность интерфейса должна поддерживать полный жизненный цикл итерационной проверки работы. Интерфейс должен обеспечивать:

1. **Для роли «Студент»:**

   – возможность регистрации и привязки профиля к уникальному идентификатору мессенджера Telegram;

   – бесконфликтную загрузку файлов в формате `.pdf` и их передачу в модуль нормоконтроля;

   – визуализацию структурированного отчета об ошибках оформления с указанием конкретных нарушенных пунктов требований к

оформлению;

– получение оперативных уведомлений об изменении статуса проверки (переход от автоматического фильтра к ручной проверке преподавателем).

2. **Для роли «Преподаватель»:**

– механизм авторизации по одноразовым токенам (кодам доступа);

– доступ к реестру работ, успешно прошедших интеллектуальный фильтр, с возможностью скачивания исходных файлов;

– функционал для фиксации финального вердикта («Принять» / «На доработку») и ввода текстовой рецензии;

– просмотр истории итераций конкретного студента для контроля исправления ранее выявленных замечаний.

Нефункциональные требования

К качественным характеристикам интерфейса и среды эксплуатации предъявляются следующие требования:

1. **Удобство и доступность** - интерфейс следует проектировать на базе мессенджера Telegram, что обеспечивает кроссплатформенность (iOS, Android, Desktop) и удобство для студентов и преподавателей без необходимости установки специализированного ПО.

2. **Скорость работы** - необходимо обеспечить время отклика интерфейса на управляющие команды не более 2 секунд. Для длительных операций (интеллектуальный анализ PDF) следует использовать асинхронную модель взаимодействия с индикацией состояния процесса.

3. **Надежность** - не допускается потеря пользовательских данных при разрыве соединения; система должна сохранять состояние диалога и позволять продолжить загрузку после восстановления связи.

4. **Расширяемость** - архитектуру интерфейсных сообщений необходимо проектировать с учетом возможности добавления новых типов проверок (например, антиплагиат) без изменения логики взаимодействия с пользователем.

Требования к «интеллектуальности» интерфейса

Интеллектуальная составляющая интерфейса реализуется через механизмы адаптивного взаимодействия и интерпретируемости результатов:

1. Интерфейс должен реализовать превентивную валидацию ввода на стороне клиента. При регистрации студента формат ФИО проверяется регулярным выражением, соответствующим шаблону «Фамилия И.О.» (кириллица, заглавная первая буква фамилии, две инициала с точками). Поле

номера группы должно автоматически фильтровать нецифровые символы и ограничивать длину. Логика валидации должна быть явно описана и воспроизводима, а не основана на эвристиках.

2. Интерфейс должен реализовать механизм адаптивной блокировки действий: кнопка отправки файла должна становиться неактивной при невыполнении обязательных предусловий (отсутствие файла или невыбранный преподаватель). Условия блокировки должны вычисляться в реальном времени при каждом изменении состояния формы.

3. Интерфейс должен обеспечивать объяснимость результатов проверки. Отчёт об ошибках должен содержать не только факт нарушения, но и конкретные параметры: номер страницы, обнаруженное значение параметра и ожидаемое значение (например, «обнаружен шрифт Arial, ожидается Times New Roman»). Объяснение должно быть достаточным для самостоятельного устранения ошибки студентом без обращения к преподавателю.

4. Интерфейс должен использовать цветовую дифференциацию статусов работ (зелёный — принято, красный — отклонено, нейтральный — в обработке) с дублированием информации текстовыми метками для обеспечения доступности.

5. Интерфейс должен предоставлять контекстные подсказки при вводе данных: описание ожидаемого формата под полями ввода и примеры корректных значений в атрибутах placeholder.

1.9 Вывод

По результатам анализа предметной области, участников, систем-аналогов и требований к интерфейсу можно сделать следующие выводы:

1. Ручная проверка оформления студенческих работ является трудоёмким и ошибкоподверженным процессом. Автоматизация контроля формальных параметров документа (шрифт, кегль, отступы) позволяет существенно сократить временные затраты преподавателя и ускорить цикл обратной связи.

2. В системе выделены две роли с принципиально различными сценариями взаимодействия: студент загружает документы и получает результаты проверки, преподаватель верифицируется по корпоративной почте и выполняет экспертное рецензирование. Это определяет необходимость раздельных процедур регистрации и разграничения доступа на уровне интерфейса и серверной части.

3. Существующие аналоги либо не предоставляют автоматической проверки форматирования (LMS Moodle, Google Classroom), либо ограничены проверкой кода и не работают с текстовыми документами. Ни одно из рассмотренных решений не интегрировано с мессенджером Telegram, что оставляет нишу для разработки специализированной системы.

4. Платформа Telegram Mini Apps выбрана в качестве среды развёртывания интерфейса, поскольку она обеспечивает встроенную аутентификацию, push-уведомления и не требует от пользователя установки дополнительного программного обеспечения. Ограничения платформы (размер экрана, возможности встроенного браузера) учтены в нефункциональных требованиях.

5. Сформулированные функциональные требования охватывают полный цикл взаимодействия пользователя с системой: от регистрации до получения отчёта о проверке. Требования к интеллектуальности интерфейса ограничены реалистичным для курсового проекта уровнем — превентивная валидация, объяснимость результатов и адаптация к теме оформления.

6. При проектировании будут использованы: распределённая архитектура с асинхронной обработкой через очередь сообщений, трёхслойная структура интерфейсного слоя, аутентификация через Telegram initData и генерация PDF-отчётов с постраничной детализацией ошибок.

2 ПРОЕКТИРОВАНИЕ ИНТЕЛЛЕКТУАЛЬНОЙ СИСТЕМЫ

2.1 Общая архитектура интеллектуальной системы

Система SAPSR построена по принципу распределённой сервис-ориентированной архитектуры и включает следующие подсистемы:

1. **Слой интерфейса (Telegram).** Объединяет два канала взаимодействия с пользователем: Telegram Mini App - графический интерфейс, реализованный как встроенное веб-приложение внутри мессенджера, и Telegram Bot API - канал асинхронных push-уведомлений о результатах проверки.

2. **Серверный слой (Backend).** Центральный оркестратор системы, реализованный на платформе Java Spring Boot. Выполняет функции маршрутизации запросов, управления бизнес-логикой, аутентификации пользователей по данным Telegram, хранения и извлечения данных через JPA/JDBC-соединение с СУБД PostgreSQL, а также постановки заданий на проверку через брокер сообщений RabbitMQ.

3. **Подсистема нормоконтроля (Python Worker).** Аналитический воркер, реализованный на языке Python. Извлекает из PDF-документа метаданные символов (гарнитура, кегль, координаты) с помощью библиотеки pdfplumber и сопоставляет их с набором правил оформления, выступающим в роли базы знаний системы. Результат анализа передаётся обратно в оркестратор через очередь сообщений.

4. **Сервер базы данных (PostgreSQL).** Хранит данные о студентах, преподавателях, загруженных документах, статусах проверок и обнаруженных ошибках форматирования.

5. **Внешние системы.** Предусмотрена точка интеграции с API ИИС БГУИР для верификации преподавательских учётных записей по корпоративной электронной почте.

Место интерфейса в архитектуре определяется его ролью связующего звена между пользователем и серверным слоем. Mini App не содержит бизнес-логики проверки - оно формирует запросы к REST API оркестратора и визуализирует полученные ответы. Telegram Bot API используется серверным слоем для отправки push-уведомлений без участия интерфейса. Архитектурная схема верхнего уровня представлена на рисунке 2.1.

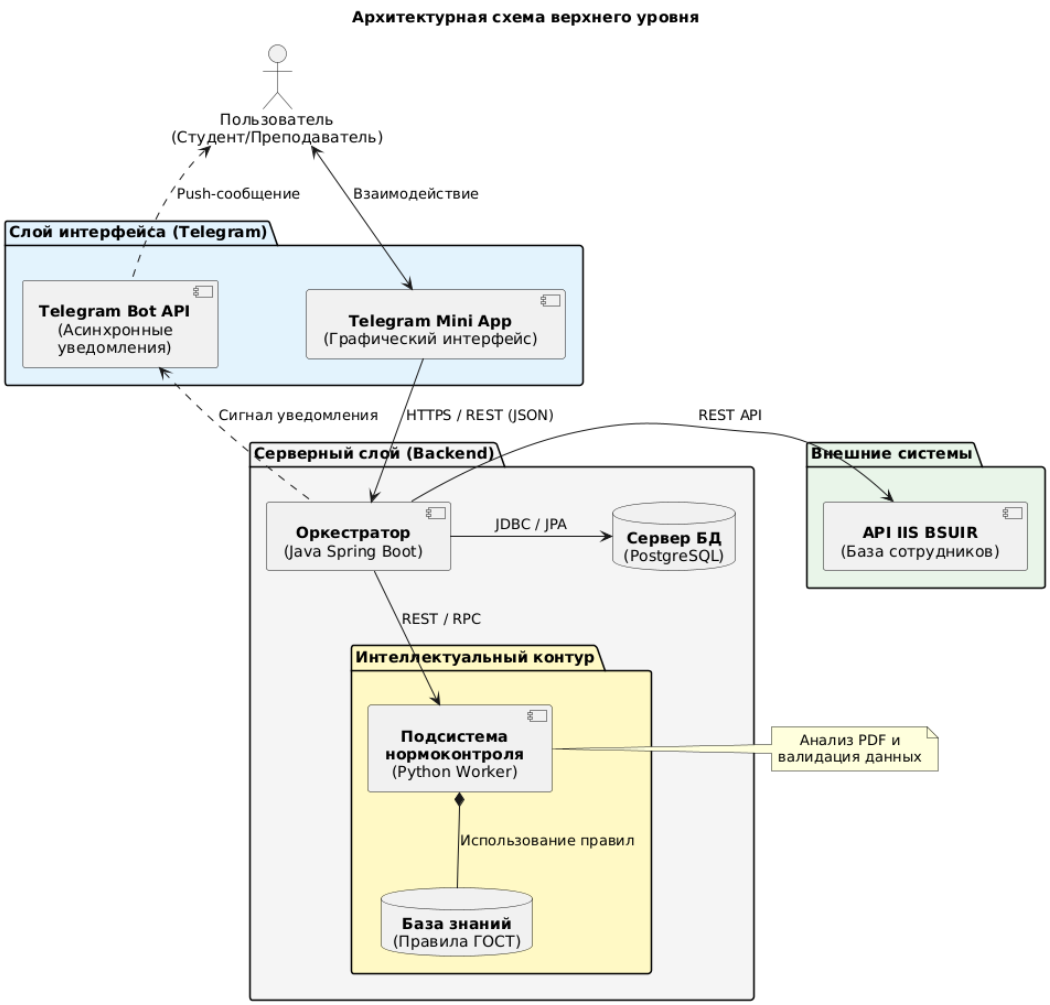

Рисунок 2.1 – Архитектурная схема верхнего уровня системы SAPSR

2.2 Архитектура интерфейсного слоя

Структура модулей и слоёв

Интерфейсный слой системы реализован как одностраничное React-приложение и разделён на три логических уровня (рисунок 2.2):

1. **Слой представления (Presentation Layer).** Отвечает за визуальное отображение данных. Реализован средствами HTML5 и CSS3 с адаптивной вёрсткой под размеры экрана Telegram Mini App. Использует библиотеку Framer Motion для анимации переходов между экранами. Слой не содержит логики обработки данных - он лишь отображает состояние, переданное из нижележащего слоя.

2. **Слой логики взаимодействия (Logic Layer).** Управляет состоянием приложения через React-хуки (useState, useEffect). Реализует клиентскую валидацию вводимых данных (формат ФИО, номер группы, корректность email), управление навигацией между экранами и вкладками, а также взаимодействие с Telegram WebApp SDK - аутентификацию, тактильную обратную связь (HapticFeedback) и системные диалоги.

3. **Коммуникационный слой (Communication Layer).** Инкапсулирует все HTTP-запросы к серверному API. Реализован через Fetch API с асинхронными вызовами. Автоматически вставляет заголовок Authorization с данными initData из Telegram SDK в каждый запрос. Обрабатывает ошибки сети и некорректные ответы сервера, транслируя их в понятные пользователю сообщения.

Границы ответственности модулей определены следующим образом: слой представления не обращается к API напрямую; слой логики не формирует HTTP-запросы - он вызывает функции коммуникационного слоя; коммуникационный слой не изменяет состояние UI - он возвращает данные, которые интерпретирует слой логики.

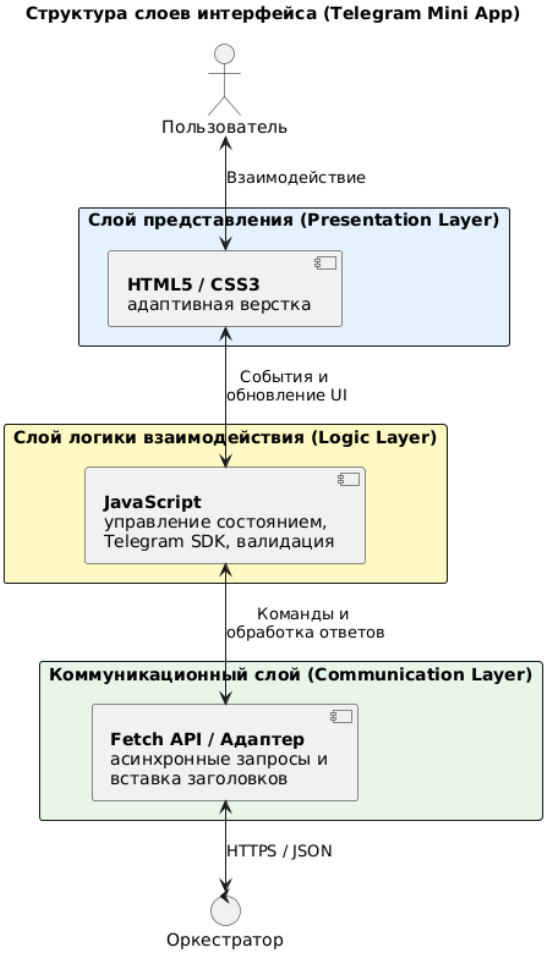

Рисунок 2.2 – Структура слоёв интерфейса Telegram Mini App

Взаимодействие с внутренними компонентами

Интерфейс взаимодействует с серверным слоем по протоколу HTTPS, обмениваясь данными в формате JSON. Все запросы проходят через единую точку входа - REST API оркестратора, доступный по базовому пути /api/v1.

Аутентификация реализована без классических логина и пароля. При каждом запросе интерфейс передаёт в заголовке Authorization стро-

ку initData, которую Telegram автоматически формирует при открытии Mini App. Серверный перехватчик (TelegramInitDataInterceptor) проверяет HMAC-подпись этих данных, используя токен бота, и извлекает идентификатор пользователя Telegram.

Механизмы обработки ошибок и аварийных ситуаций:

– **Недоступность сервера.** При ошибке сети (catch блок Fetch API) интерфейс отображает сообщение «Сервер недоступен» без аварийного завершения работы.

– **Ошибка аутентификации (HTTP 401/403).** Пользователь перенаправляется на экран выбора роли для повторной регистрации.

– **Ошибка валидации (HTTP 400).** Сервер возвращает JSON с полем error, содержащим текстовое описание на русском языке. Интерфейс отображает его в области ошибок соответствующей формы.

– **Внутренняя ошибка сервера (HTTP 500).** Отображается общее сообщение об ошибке с предложением повторить попытку.

2.3 Модели пользователей и сценарии использования

Ролевые модели пользователей и внешних систем

В системе выделены две пользовательские роли и одна внешняя система (рисунок 2.3):

1. **Студент.** Основной пользователь системы. Регистрируется путём ввода ФИО в формате «Фамилия И.О.» и шестизначного номера учебной группы. После регистрации получает доступ к загрузке PDF-документов на проверку, просмотру истории отправленных работ, скачиванию отчётов об ошибках форматирования. При загрузке файла обязан выбрать преподавателя-получателя из предзаполненного списка. Ограничения: может видеть только свои работы, не имеет доступа к работам других студентов.

2. **Преподаватель.** Верифицируется через корпоративную почту @bsuir.by. Список допустимых преподавателей предзаполнен в базе данных. При первом входе система отправляет шестизначный код подтверждения на указанную почту, после ввода которого аккаунт Telegram привязывается к записи преподавателя. Повторная регистрация не требуется. Преподаватель получает уведомления о прошедших автоматическую проверку работах и может выполнять экспертное рецензирование.

3. **API ИИС БГУИР (внешняя система).** Источник данных о преподавательском составе. Используется для предзаполнения таблицы преподавателей корректными адресами электронной почты и ФИО.

Сценарии использования

Основной сценарий работы студента представлен на диаграмме последовательности (рисунок 2.4) и включает три фазы.

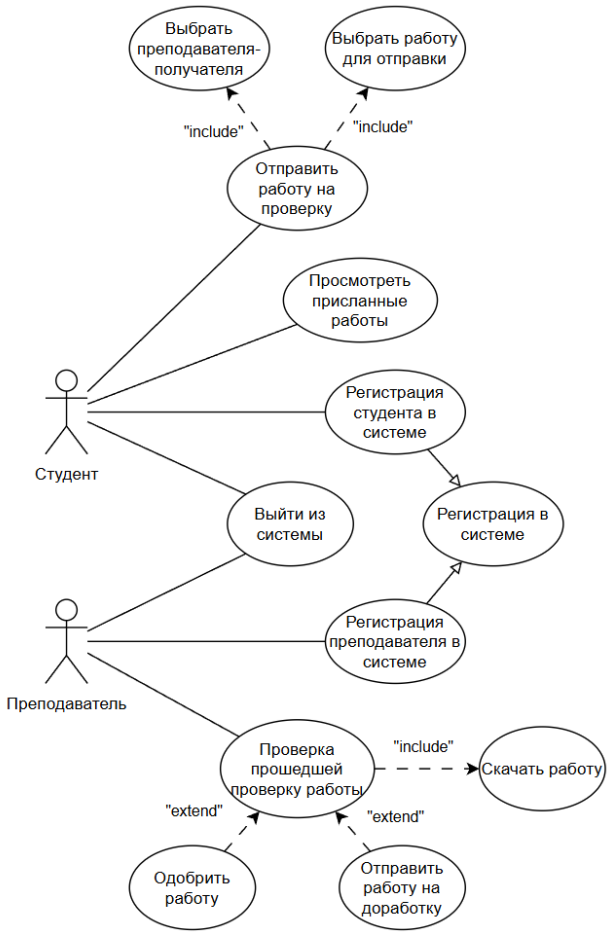

Рисунок 2.3 – Диаграмма вариантов использования системы SAPSR

Фаза 1 - Авторизация и верификация. Студент открывает Mini App через Telegram. Интерфейс автоматически передаёт данные авторизации (initData) оркестратору, который запрашивает роль и данные профиля из базы данных. Если пользователь не зарегистрирован, отображается экран выбора роли и форма регистрации. Альтернативная ветвь: если данные initData некорректны, отображается ошибка авторизации.

Фаза 2 - Загрузка и автоматический анализ. Студент выбирает преподавателя-получателя из выпадающего списка, прикрепляет PDF-файл и нажимает «Отправить». Оркестратор сохраняет файл, создаёт запись в базе данных со статусом «PROCESSING» и ставит задание в очередь RabbitMQ. Аналитический воркер извлекает задание, анализирует документ и возвращает результат. Оркестратор обновляет статус и отправляет push-уведомление студенту через Telegram Bot API. Альтернативная ветвь: если файл не является PDF или пуст, сервер возвращает ошибку валидации, интерфейс отображает её без отправки в очередь.

Фаза 3 - Экспертное рецензирование. Если автоматическая проверка пройдена успешно, преподаватель получает уведомление. Он просматривает реестр работ, скачивает документ, изучает содержание и выносит итоговое

решение - одобрить или отправить на доработку. Результат сохраняется в базе, студент уведомляется через бота.

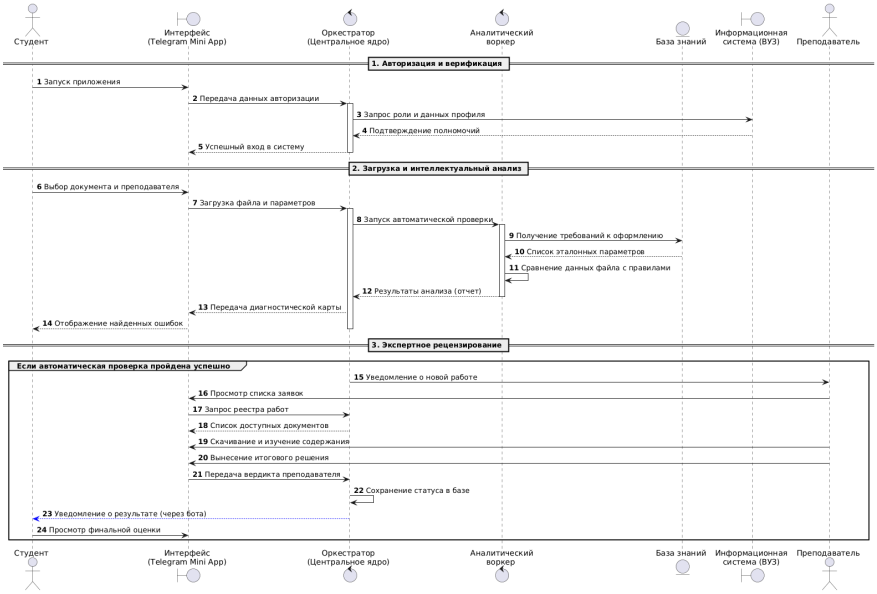

Рисунок 2.4 – Диаграмма последовательности взаимодействия компонентов системы

Диаграмма состояний студенческой работы (рисунок 2.5) отражает жизненный цикл документа в системе. Из начального состояния «В обработке» работа переходит либо в «Отклонено» (при обнаружении ошибок форматирования), либо в «На проверке» (при успешном прохождении нормоконтроля). Из состояния «Отклонено» студент может повторно загрузить исправленный документ. Из состояния «На проверке» преподаватель переводит работу в «Утверждено» или возвращает в «Отклонено».

Рисунок 2.5 – Диаграмма состояний студенческой работы

2.4 Проектирование пользовательского интерфейса

Информационная структура и навигация

Навигация в приложении организована по двухуровневой схеме. Первый уровень - линейная последовательность экранов регистрации: выбор роли, ввод данных, подтверждение. Второй уровень - основной рабочий экран с тремя вкладками, переключаемыми через нижнюю панель навигации.

Карта экранов приложения:

1. **Экран загрузки** - отображается при инициализации, пока интерфейс проверяет наличие сохранённой сессии через запрос GET /api/v1/me.

2. **Экран выбора роли** - две кнопки: «Студент» и «Преподаватель». Отображается при первом входе или после выхода из системы.

3. **Экран регистрации студента** - два поля ввода: ФИО (формат «Фамилия И.О.») и номер группы (6 цифр). Клиентская валидация выполняется до отправки на сервер.

4. **Экран регистрации преподавателя** - поле ввода корпоративной почты @bsuir.by.

5. **Экран подтверждения кода** - поле ввода шестизначного кода верификации, отправленного на почту преподавателя.

6. **Вкладка «Загрузка»** - выпадающий список преподавателей, область выбора PDF-файла, кнопка отправки.

7. **Вкладка «Уведомления»** - хронологический список загруженных работ с отображением статуса, имени файла, имени преподавателя и кнопкой скачивания PDF-отчёта.

8. **Вкладка «Выход»** - подтверждение выхода из системы.

Переходы между вкладками основного экрана анимированы горизонтальным сдвигом с пружинной интерполяцией, что соответствует привычной модели навигации в мобильных приложениях.

Макеты экранов и элементы управления

Интерфейс использует следующие элементы управления:

– **Карточки ролей** - кнопки с иконкой и текстом, визуально выделяющиеся при нажатии изменением фона на акцентный цвет Telegram-темы.

– **Текстовые поля ввода** - стилизованы с рамкой акцентного цвета, поддерживают атрибуты inputMode для вызова цифровой клавиатуры на мобильных устройствах.

– **Выпадающий список (select)** - кастомный селект для выбора преподавателя с опцией-заглушкой «Выберите преподавателя» по умолчанию.

– **Область загрузки файла** - стилизованный label, визуально оформленный как дропзона с пунктирной рамкой и иконкой. Скрытый input[type=file] с фильтром accept=".pdf".

– **Основная кнопка действия (submit)** - полноширинная, с акцентным цветом темы. Переходит в состояние disabled при отсутствии обязательных данных или во время отправки (с текстовой индикацией «Отправка...»).

– **Карточки уведомлений** - каждая содержит имя файла, имя преподавателя, статус (с цветовой индикацией) и кнопку скачивания отчёта. Карточки с ошибками выделяются красной левой границей, успешные - зелёной.

– **Панель навигации** - фиксированная внизу экрана, с полупрозрачным фоном и размытием (backdrop-filter: blur). Активная вкладка подсвечивается акцентным цветом.

Визуальная иерархия построена на использовании CSS-переменных Telegram-темы (-tg-theme-*), что обеспечивает корректное отображение как в светлой, так и в тёмной теме мессенджера.

2.5 Проектирование программного интерфейса

Спецификация операций и сообщений

REST API оркестратора доступен по базовому пути `/api/v1`. Взаимодействие с системой реализовано через следующие группы эндпоинтов:

Аутентификация и профиль: – GET /me — Получение профиля текущего пользователя.

– POST /auth/send-code — Отправка кода верификации. Тело: {email}.

– POST /auth/verify-code — Подтверждение кода и привязка Telegram. Тело: {email, code}.

Регистрация и работа с данными: – POST /register/student — Регистрация студента. Тело: {full_name, group_number}.

– GET /teachers — Получение списка всех преподавателей.

– POST /upload — Загрузка PDF-файла. Используется multipart/form-data (поля: `file`, teacher_id).

История и отчетность: – GET /submissions — Просмотр истории работ текущего студента.

– GET /submissions/{id}/report — Скачивание бинарного PDF-отчёта (Content-Type: application/pdf).

Все ответы возвращаются в формате JSON (UTF-8). Система использует стандартные коды состояния HTTP:

– **200 OK** (успех);
– **400 Bad Request** (ошибка валидации);
– **401 Unauthorized** (ошибка авторизации);
– **403 Forbidden** (доступ запрещен);
– **404 Not Found** (ресурс не найден);

– **500 Internal Server Error** (критическая ошибка).

Взаимодействие оркестратора с аналитическим воркером осуществляется через брокер сообщений RabbitMQ:

– **pdf_tasks_queue** — публикация задания: {"task_id": int, "file_path": string, "status": "PROCESSING"};

– **pdf_results_queue** — получение результата: {"task_id": int, "status": "SUCCESS"FAIL "errors": [{"page": int, "message": string}]}.

Контракты и протоколы взаимодействия

– **HTTPS / REST** — взаимодействие интерфейса с оркестратором. Аутентификация через заголовок Authorization с HMAC-подписью данных Telegram initData.

– **AMQP 0-9-1** — взаимодействие с воркером через RabbitMQ. Сообщения в JSON (UTF-8), режим доставки — persistent (delivery_mode=2).

– **JDBC** — взаимодействие с СУБД PostgreSQL через Hibernate. Режим ddl-auto=validate обеспечивает строгую проверку схемы.

– **Telegram Bot API (HTTPS)** — отправка уведомлений и управление Menu Button. Получение сообщений через long polling.

Текстовые данные передаются в кодировке UTF-8, временные метки — в формате ISO 8601, идентификаторы Telegram — тип long (64 бита).

2.6 Проектирование аппаратного интерфейса

Система SAPSR не предполагает прямого взаимодействия с аппаратными устройствами. Все компоненты функционируют в программной среде: серверная часть развёртывается в Docker-контейнерах, клиентская часть работает во встроенном браузере мессенджера Telegram. Данный подраздел неприменим к рассматриваемой системе.

2.7 Проектирование интеллектуальных механизмов интерфейса

Интеллектуальная поддержка пользователя

Интерфейс реализует следующие механизмы поддержки пользователя:

1. **Превентивная валидация ввода.** При регистрации студента интерфейс проверяет формат ФИО регулярным выражением до отправки на сервер. Поле номера группы автоматически фильтрует нецифровые символы и ограничивает длину шестью знаками. Это исключает отправку заведомо некорректных данных и сокращает количество обращений к серверу.

2. **Контекстные подсказки.** Под каждым полем ввода отображается подсказка с ожидаемым форматом данных (например, «ФИО: Фамилия

И.О. | Группа: 6 цифр»). Поля содержат атрибут placeholder с примером корректного значения.

3. **Адаптивная блокировка действий.** Кнопка отправки файла переходит в неактивное состояние, если не выполнены обязательные условия: не выбран преподаватель или не прикреплён файл. Это предотвращает ошибочные отправки без необходимости показывать дополнительные диалоги.

4. **Тактильная обратная связь.** При успешном выполнении операций (регистрация, отправка файла, скачивание отчёта) интерфейс вызывает HapticFeedback.notificationOccurred из Telegram SDK, обеспечивая вибрационный отклик на мобильных устройствах.

Объяснимость и адаптация интерфейса

Механизмы объяснимости результатов проверки:

1. **Структурированный отчёт.** При обнаружении ошибок форматирования система формирует PDF-отчёт, в котором каждая ошибка содержит номер страницы, описание нарушения и ожидаемое значение параметра (например, «Стр. 3: Неверный размер шрифта: 12.0 пт (ожидается 14.0 пт)»). Это позволяет студенту точно локализовать и устранить каждое нарушение.

2. **Цветовая индикация статусов.** В списке уведомлений работы визуально дифференцированы: зелёная граница для принятых, красная - для отклонённых, нейтральная - для находящихся в обработке. Текстовые метки статусов дублируют цветовую информацию для обеспечения доступности.

3. **Адаптация к теме оформления.** Интерфейс использует CSS-переменные Telegram-темы (-tg-theme-bg-color, -tg-theme-text-color, -tg-theme-button-color и другие), автоматически подстраиваясь под светлую или тёмную тему, выбранную пользователем в настройках мессенджера. Дополнительных действий от пользователя не требуется.

2.8 Вывод по разделу проектирования

В ходе проектирования системы SAPSR приняты следующие архитектурные решения:

1. Выбрана распределённая архитектура с разделением на интерфейсный слой (Telegram Mini App), серверный оркестратор (Spring Boot), аналитический воркер (Python) и СУБД (PostgreSQL). Асинхронное взаимодействие между оркестратором и воркером через RabbitMQ обеспечивает масштабируемость и отказоустойчивость подсистемы анализа.

2. Интерфейсный слой разделён на три уровня ответственности (представление, логика, коммуникация), что упрощает сопровождение и тестирование каждого уровня независимо.

3. Аутентификация реализована через встроенный механизм Telegram initData с проверкой HMAC-подписи, что исключает необходимость хранения паролей и управления сессиями.

4. Разделение пользователей на две роли (студент и преподаватель) с различными сценариями регистрации и разными наборами доступных операций обеспечивает выполнение требований к разграничению доступа, сформулированных в разделе анализа.

5. Предусмотрены возможности расширения: добавление новых правил проверки в базу знаний воркера без изменения оркестратора и интерфейса; подключение дополнительных аналитических воркеров к очереди сообщений для горизонтального масштабирования; интеграция с внешними информационными системами вуза через REST API.

3 РЕАЛИЗАЦИЯ ИНТЕЛЛЕКТУАЛЬНОЙ СИСТЕМЫ

3.1 Выбор средств реализации

Языки программирования, фреймворки и библиотеки
Обоснование выбора средств
Реализация пользовательского интерфейса

3.2 Реализация программного (или аппаратного) интерфейса

3.3 Тестирование интерфейса

Функциональное и интеграционное тестирование
Оценка удобства и качества взаимодействия

3.4 Вывод по разделу реализации

ЗАКЛЮЧЕНИЕ

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] EssayGrader. EssayGrader: AI-Powered Essay Grading and Feedback Tool. — 2024. — Mode of access: https://www.essaygrader.ai/. — Date of access: 22-05-2024.

[2] Gradescope by Turnitin. Gradescope: Streamline and standardize the grading of paper-based, digital, and coding assignments. — 2024. — Mode of access: https://www.gradescope.com/. — Date of access: 22-05-2024.

[3] EduMark. Официальный сайт платформы EduMark. — 2026. https://edumark.ru.

[4] Документация Python [Электронный ресурс]. — Режим доступа: https://docs.python.org/3/. — Дата доступа: 10.03.2025.

[5] Arnold, Ken. The Java programming language / Ken Arnold, James Gosling, David Holmes. — Addison Wesley Professional, 2005.

[6] PostgreSQL: The World's Most Advanced Open Source Relational Database [Electronic resource]. — Mode of access: https://www.postgresql.org/. — Date of access: 26.02.2026.

[7] Docker Documentation [Electronic resource]. — Mode of access: https://docs.docker.com/. — Date of access: 26.02.2026.

[8] RabbitMQ Documentation [Electronic resource]. — Mode of access: https://www.rabbitmq.com/docs. — Date of access: 26.02.2026.